Guide illustré

SIL — Software-In-the-Loop du calculateur d'éclairage

Projet Lighting System Control — Amina Borcheni — M2 IAA, ENET'Com — MATLAB R2022b (MATLAB Coder + Embedded Coder)

Objectif : après le MIL (modèle testé), on vérifie maintenant le **logiciel réel** : le code C généré automatiquement à partir des 8 blocs validés. Le code C est compilé, exécuté pas à pas sur les mêmes scénarios que l'intégration, et chaque sortie est comparée à celle du modèle (test **back-to-back**).

Niveau de test	Ce qu'on teste	Où	Résultat
MIL — unitaire	Chaque bloc (modèle Simulink)	MATLAB/Simulink	113 / 113 PASS
MIL — intégration	Les 8 blocs reliés, 4 scénarios	MATLAB/Simulink	36 / 36 PASS
SIL	**Le code C généré du calculateur complet**	**PC (code C compilé)**	**6 / 6 PASS — 27 004 pas, 0 écart**
PIL / HIL (suite)	Code sur le microcontrôleur réel	Carte STM32 / Arduino	perspective

1. Qu'est-ce que le SIL ?

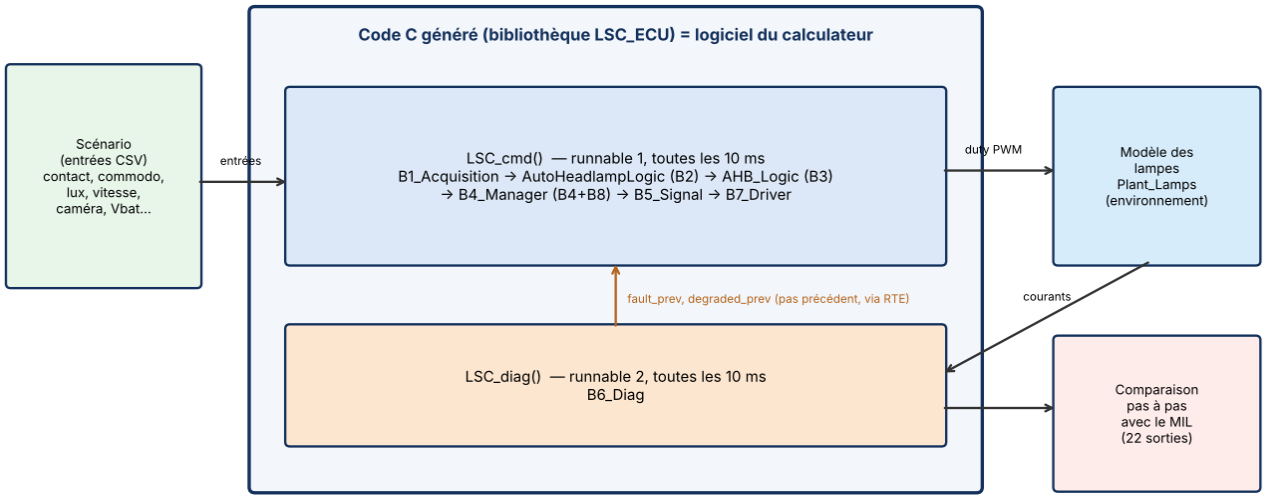
Dans le cycle en V, chaque niveau de la branche descendante a son test sur la branche montante. Le MIL vérifie que **l'algorithme** est juste. Mais le calculateur n'exécute pas un modèle Simulink : il exécute du **code C**. Le SIL vérifie que la traduction modèle → code C n'a rien changé : types (uint8, boolean), arrondis, saturations, ordre d'exécution, mémoires (variables persistent devenues static).

	MIL	SIL	HIL
Ce qui tourne	Le modèle	Le code C compilé sur PC	Le code C sur le vrai calculateur
Environnement	Simulé	Simulé (banc de test)	Simulé en temps réel + vrai matériel
Erreurs trouvées	Erreurs de logique	Erreurs de génération de code, de types, de précision	Erreurs de timing, d'électronique

Question du jury : « Pourquoi le SIL est-il utile si le code est généré automatiquement ? » — Le générateur peut changer le comportement : conversion double → uint8 des PWM, comparaisons en virgule flottante, initialisation des mémoires. ISO 26262-6 demande de vérifier le code par des tests « back-to-back » entre modèle et code.

2. L'architecture du logiciel en C

Les 8 blocs sont regroupés en **deux runnables** appelés toutes les 10 ms, comme dans un composant AUTOSAR. Le modèle des lampes n'est pas du logiciel embarqué : il reste dans le banc de test, qui joue le rôle du véhicule et du RTE (les données échangées entre runnables sont celles du pas précédent).



Architecture du SIL : code C généré au centre, banc de test autour

Runnable	Blocs	Fichier MATLAB	Rôle
LSC_cmd()	B1 → B2 → B3 → B4/B8 → B5 → B7	LSC_cmd.m	Acquisition, décisions, pilotage PWM des lampes
LSC_diag()	B6	LSC_diag.m	Diagnostic des lampes, DTC, mode dégradé

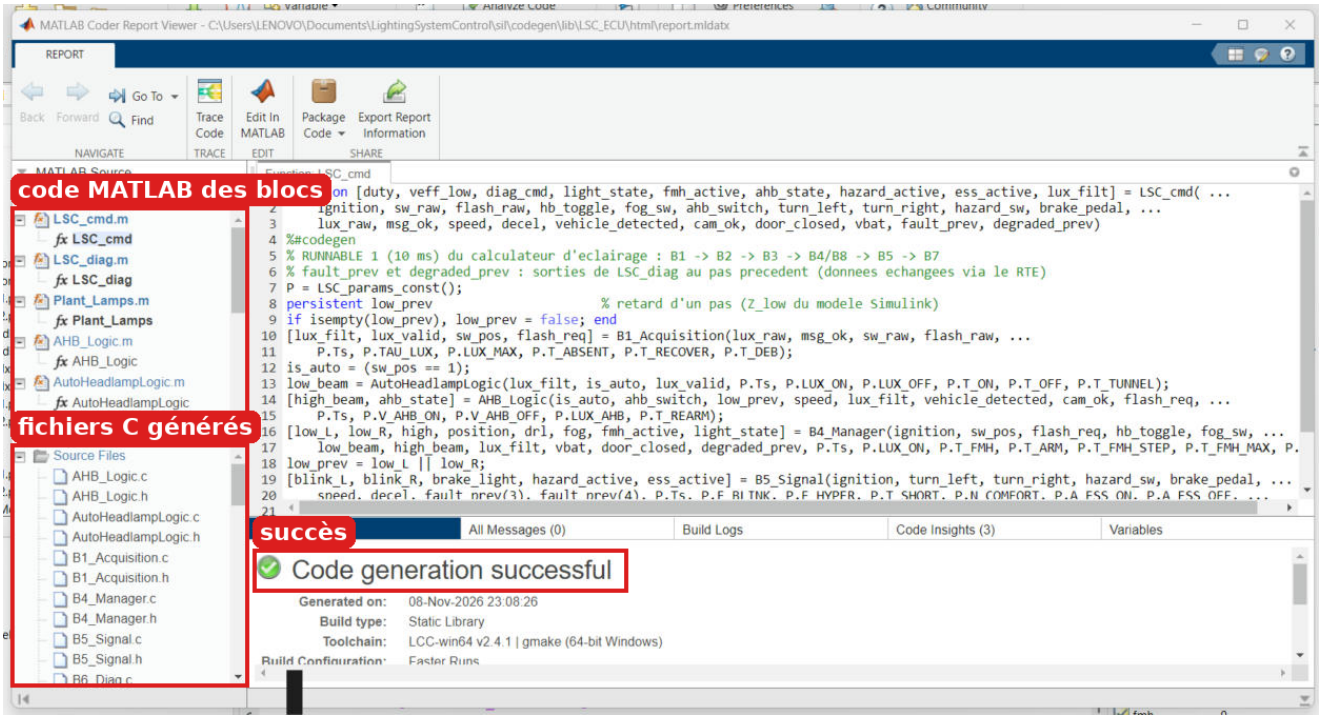
Les boucles de rétroaction sont gérées comme dans le modèle Simulink : **low_prev** (Z_low) est une variable mémoire de LSC_cmd ; **fault_prev** et **degraded_prev** (Z_fault, Z_degraded) sont les sorties de LSC_diag du pas précédent, transmises par le banc de test (rôle du RTE).

3. Lancer le SIL

1. Dans MATLAB, dossier **Documents\LightingSystemControl**, tape **sil_all** puis Entrée (≈ 2 min).
2. Le script copie le code validé de chaque bloc (**scripts\Bx_logic.txt**) dans le dossier **sil**, écrit les paramètres de calibration en constantes (**LSC_params_const.m**).
3. Il exécute les 4 scénarios avec les runnables en MATLAB et vérifie que le résultat est **identique au modèle Simulink LSC_Integration** (test SIL-00).
4. Il génère le code C avec Embedded Coder dans **sil\codegen\lib\LSC_ECU** et exporte les entrées et sorties de référence (**sil\S1..S4_in.csv** et **_mil.csv**).
5. Le banc de test **sil\sil_harness.c** compile le code C, l'exécute sur chaque scénario et compare les 22 sorties à chaque pas (compilation : **gcc -std=c89 -o sil_ecu *.c**, puis **sil_ecu S1_in.csv S1_mil.csv S1_sil.csv**). Résultat dans **results\SIL_resultats.txt**.

4. La génération du code C

Réglage	Valeur	Pourquoi
Outil	MATLAB Coder 5.5 + Embedded Coder 7.9	Code embarqué lisible, sans allocation dynamique
Type de sortie	Bibliothèque statique (lib)	Le code est intégré dans le logiciel du calculateur
Langage	C89/90 (ANSI)	Compatible avec tous les compilateurs embarqués
Paramètres	Constantes (coder.Constant)	Les seuils (1000 lux, 2 s...) sont figés dans le code
Mémoires	persistent → variables static	L'état de chaque bloc est conservé entre deux appels



Rapport de génération (sil\codegen\lib\LSC_ECU\html\report.mldatx) : à gauche le code MATLAB et les fichiers C, au centre LSC_cmd.m

Chaque bloc devient un fichier C : on retrouve l'architecture du projet dans le code.

Bloc	Fichier C	Instructions C	Taille compilée
B1 Acquisition	B1_Acquisition.c	46	1,2 ko
B2 Allumage auto	AutoHeadlampLogic.c	30	1,1 ko
B3 AHB	AHB_Logic.c	29	0,6 ko
B4 + B8 Manager / follow-me-home	B4_Manager.c	97	2,9 ko
B5 Signalisation	B5_Signal.c	75	2,2 ko
B6 Diagnostic	B6_Diag.c	144	2,6 ko
B7 Pilotage	B7_Driver.c	48	1,4 ko
Runnables	LSC_cmd.c, LSC_diag.c	25	1,5 ko
Total logiciel (avec fonctions utilitaires)			**≈ 16 ko**

Tailles mesurées avec gcc -O2 sur PC (x86-64), à titre indicatif : sur un microcontrôleur ARM, elles seront différentes.

Du code MATLAB au code C : exemple de B2



AutoHeadlampLogic.c dans le rapport : la logique d'hystérésis de B2 en C

Le code C reprend exactement la logique MATLAB : les paramètres sont devenus des nombres (1000.0, 2.005 = T_ON + Ts/2), les commentaires du code MATLAB sont recopiés. Le runnable LSC_cmd appelle les blocs dans l'ordre :

```
B1_Acquisition(lux_raw, msg_ok, sw_raw, flash_raw, lux_filt, &low_beam, &sw_pos, &flash_req);
is_auto  = (sw_pos == 1.0);
low_beam = AutoHeadlampLogic(*lux_filt, is_auto, low_beam);
AHB_Logic(is_auto, ahb_switch, low_prev, speed, *lux_filt, ..., &high_beam, ahb_state);
B4_Manager(ignition, sw_pos, flash_req, ..., degraded_prev, &low_L, &low_R, ...);
low_prev = (low_L || low_R);           /* memoire Z_low pour le pas suivant */
B5_Signal(ignition, ..., fault_prev[2], fault_prev[3], ...);
B7_Driver(low_L, low_R, high, ..., vbat, duty, veff_low, diag_cmd);
```

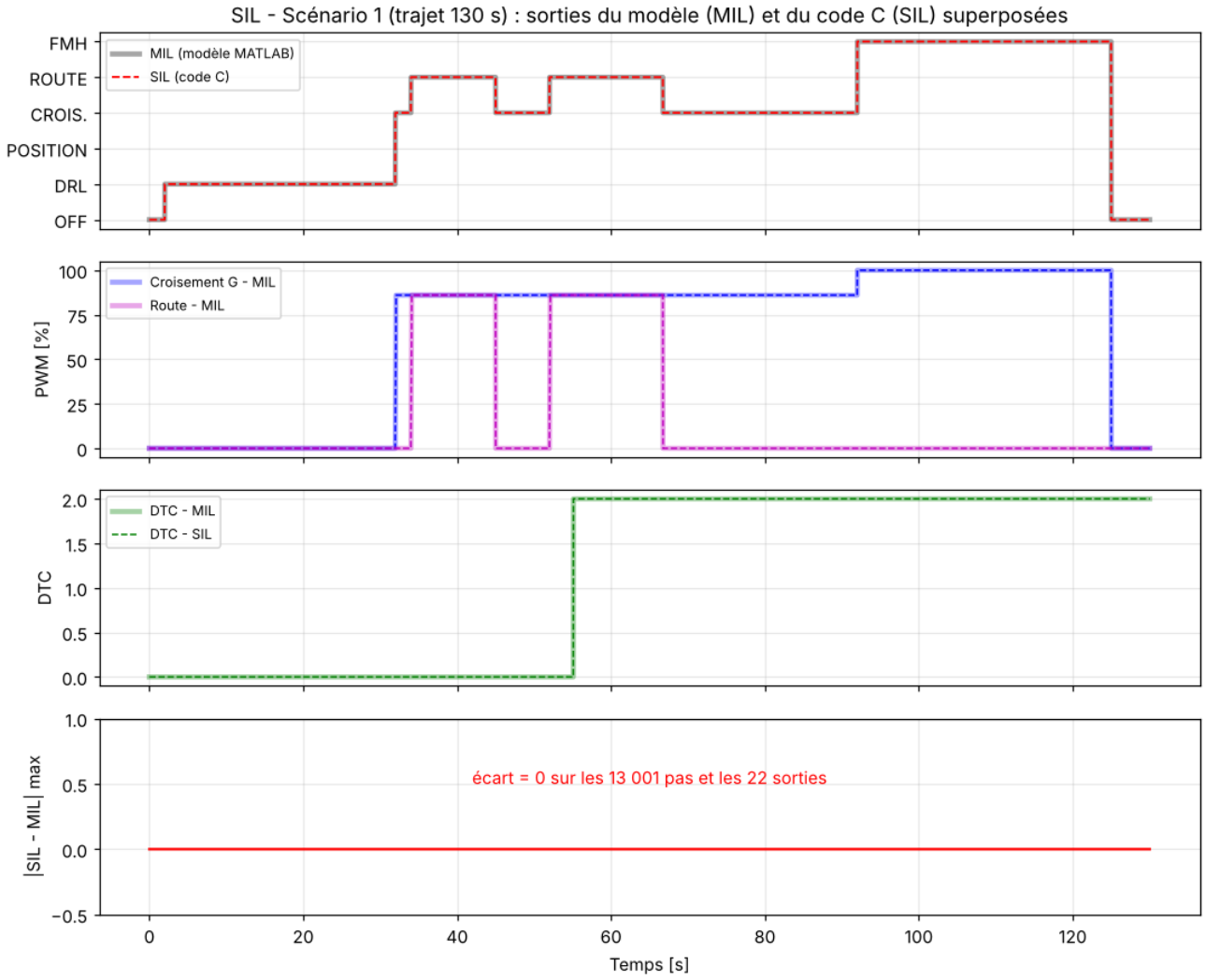
Remarque : le générateur réutilise certaines variables pour économiser la mémoire (par exemple **low_beam** contient d'abord lux_valid). C'est normal dans du code généré ; la traçabilité est assurée par le rapport (bouton **Trace Code**).

5. Le test back-to-back

Le banc de test **sil_harness.c** lit le scénario ligne par ligne (un pas de 10 ms), appelle LSC_cmd, calcule les courants des lampes (Plant_Lamps), appelle LSC_diag, puis compare les **22 sorties** au résultat MIL :

Sorties comparées	Type	Critère
8 PWM (duty), état des feux, état AHB, degraded	uint8	Égalité exacte
FMH, détresse, ESS, témoin, 4 défauts lampes	boolean	Égalité exacte
DTC mémorisés	uint16	Égalité exacte
Luminosité filtrée, tension efficace	double	Écart relatif ≤ 10^{-12}

Test	Vérification	Pas	Écarts	Résultat
SIL-00	Les runnables MATLAB reproduisent le modèle Simulink LSC_Integration (scénario 1)	13 001	0	PASS
SIL-01	Scénario 1 : trajet complet (130 s)	13 001	0	PASS
SIL-02	Scénario 2 : tunnel en plein jour (40 s)	4 001	0	PASS
SIL-03	Scénario 3 : panne de caméra + appel de phares (40 s)	4 001	0	PASS
SIL-04	Scénario 4 : batterie faible en follow-me-home (60 s)	6 001	0	PASS
SIL-05	Test de mutation : le banc détecte une erreur introduite dans le code C	13 001	65 détectés	PASS



SIL_S1.png : courbes MIL (épaisses) et SIL (pointillés) parfaitement superposées ; l'écart reste à 0

Conséquence : comme le code C donne exactement les mêmes sorties que le modèle à chaque pas, **les 36 tests d'intégration (INT-01 à 18, S2 à S4) sont aussi valides pour le code C**.

6. Le test de mutation : prouver que le banc de test fonctionne

Un test qui ne trouve jamais d'erreur peut aussi être un test qui ne vérifie rien. On a donc volontairement introduit une erreur dans le code C : le seuil d'allumage de B2 passe de **1000 à 500 lux**.

Scénario	Résultat avec le code faussé	Explication
1 — trajet	**65 pas différents** sur 8 sorties (état, PWM…)	Au crépuscule, le croisement s'allume plus tard : le banc le voit
2 — tunnel	0 différence	Dans le tunnel il y a 150 lux, sous les deux seuils : l'erreur n'a pas d'effet ici

Ce résultat montre aussi l'intérêt d'**avoir plusieurs scénarios** : une erreur invisible dans un scénario est détectée dans un autre. Le code C d'origine a ensuite été remis : tous les tests repassent.

7. Bilan et traçabilité

Niveau	Tests	Résultat
MIL unitaire (B1 à B8)	113	113 PASS
MIL intégration (4 scénarios)	36	36 PASS
SIL (code C du calculateur complet)	6	6 PASS
Total	**155**	**155 PASS**

Les 32 exigences du cahier des charges sont couvertes par les tests MIL ; le SIL prouve que le **code C** respecte les mêmes exigences, puisqu'il reproduit le modèle sur les 4 scénarios.

Limites (à dire honnêtement au jury)

Limite	Explication	Suite
Compilé sur PC	Le code C a été compilé avec gcc (C89) sur un PC Linux : ce n'est pas encore le microcontrôleur de la voiture	PIL / HIL sur carte STM32
SIL hors de MATLAB	Le compilateur LCC de MATLAB n'a pas réussi la compilation sur ton PC Windows ; le banc est donc un programme C séparé, qui lit les mêmes données	Installer MinGW-w64 (gratuit) : gcc -std=c89 -o sil_ecu *.c
Code non AUTOSAR	Le code est généré par MATLAB Coder : pas d'API RTE (Rte_Read/Rte_Write) ni de fichier ARXML	Simulink Coder + AUTOSAR Blockset
Pas de couverture de code	On n'a pas mesuré quelles lignes du code C sont exécutées (MC/DC)	Simulink Coverage ou gcov

8. Questions probables du jury

- **Quelle est la différence entre MIL et SIL ?**
  En MIL on teste le modèle, en SIL le code C généré, compilé et exécuté. Le SIL trouve les erreurs de génération de code et de types.
- **Comment savez-vous que le code C est correct ?**
  Test back-to-back : 27 004 pas sur 4 scénarios, 22 sorties comparées à chaque pas, 0 écart. Et le test de mutation prouve que le banc détecte une erreur.
- **Pourquoi deux runnables ?**
  Pour séparer la commande des lampes et le diagnostic, comme deux runnables d'un composant AUTOSAR. Les données entre les deux passent par le RTE avec un pas de retard, comme dans le modèle.
- **Pourquoi le modèle des lampes n'est pas dans le code C du calculateur ?**
  Ce n'est pas du logiciel embarqué : il représente l'ampoule réelle. Il fait partie de l'environnement de test.
- **Pourquoi C89 ?**
  C'est la norme la plus portable : tous les compilateurs de microcontrôleurs l'acceptent. MISRA C:2012 accepte aussi C90.
- **Que faudrait-il pour un vrai calculateur ?**
  Générer le code AUTOSAR (Simulink Coder + AUTOSAR Blockset) avec les API RTE et l'ARXML, puis tester en PIL/HIL sur la cible, et mesurer la couverture et le temps d'exécution.

Erreurs fréquentes

Problème	Solution
« Undefined function B1_Acquisition »	Lancer sil_all depuis le dossier du projet : il copie les blocs dans sil\
« tmwtypes.h : No such file »	Copier matlabroot\extern\include\tmwtypes.h dans le dossier du code (sil_all le fait)
Écarts SIL / MIL sur les valeurs analogiques	Exporter les CSV avec 17 chiffres (%.17g), sinon les arrondis changent les comparaisons de seuil
« Simulink Coder is required »	Normal : on génère avec MATLAB Coder à partir du code des blocs, pas depuis le modèle Simulink